



Ho Chi Minh City University of Technology
Faculty of Computer Science and Engineering

Chapter 1: Fundamentals

Algorithms Analysis and Design (C03031)

Instructor: Assoc. Prof. Dr. Vo Thi Ngoc Chau

Email: chauvtn@hcmut.edu.vn



Course Learning Outcomes

1. Able to analyze the complexity of the algorithms (recursive or iterative) and estimate the efficiency of the algorithms.
2. Improve the ability to design algorithms in different areas *by decomposing a computing problem into advanced algorithm design strategies.*
3. Able to discuss NP-completeness.



References

- [1] Cormen, T. H., Leiserson, C. E, and Rivest, R. L., *Introduction to Algorithms*, The MIT Press, 2009.
- [2] Levitin, A., *Introduction to the Design and Analysis of Algorithms*, 3rd Edition, Pearson, 2012.
- [3] Sedgewick, R., *Algorithms in C++*, Addison-Wesley, 1998.
- [4] Weiss, M.A., *Data Structures and Algorithm Analysis in C*, The Benjamin/Cummings Publishing, 1993.



Course Outline

- ❑ **Chapter 1. Fundamentals**
- ❑ Chapter 2. Divide-and-Conquer Strategy
- ❑ Chapter 3. Decrease-and-Conquer Strategy
- ❑ Chapter 4. Transform-and-Conquer Strategy
- ❑ Chapter 5. Dynamic Programming and Greedy Strategies
- ❑ Chapter 6. Backtracking and Branch-and-Bound
- ❑ Chapter 7. NP-completeness
- ❑ Chapter 8. Approximation Algorithms



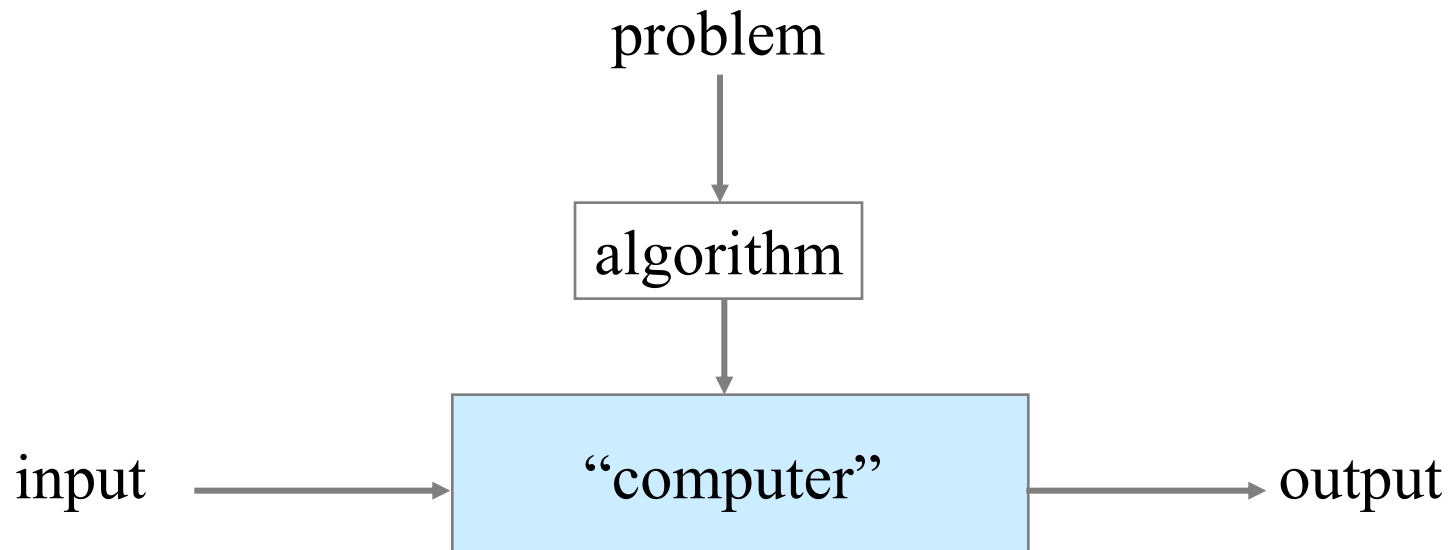
Chapter 1. Fundamentals

- 1.1. Recursion and recurrence relations
- 1.2. Analysis of algorithms
- 1.3. Analysis of iterative algorithms
- 1.4. Analysis of recursive algorithms
- 1.5. Algorithm design strategies
- 1.6. Brute-force algorithm design

Quick Review

□ What is an algorithm?

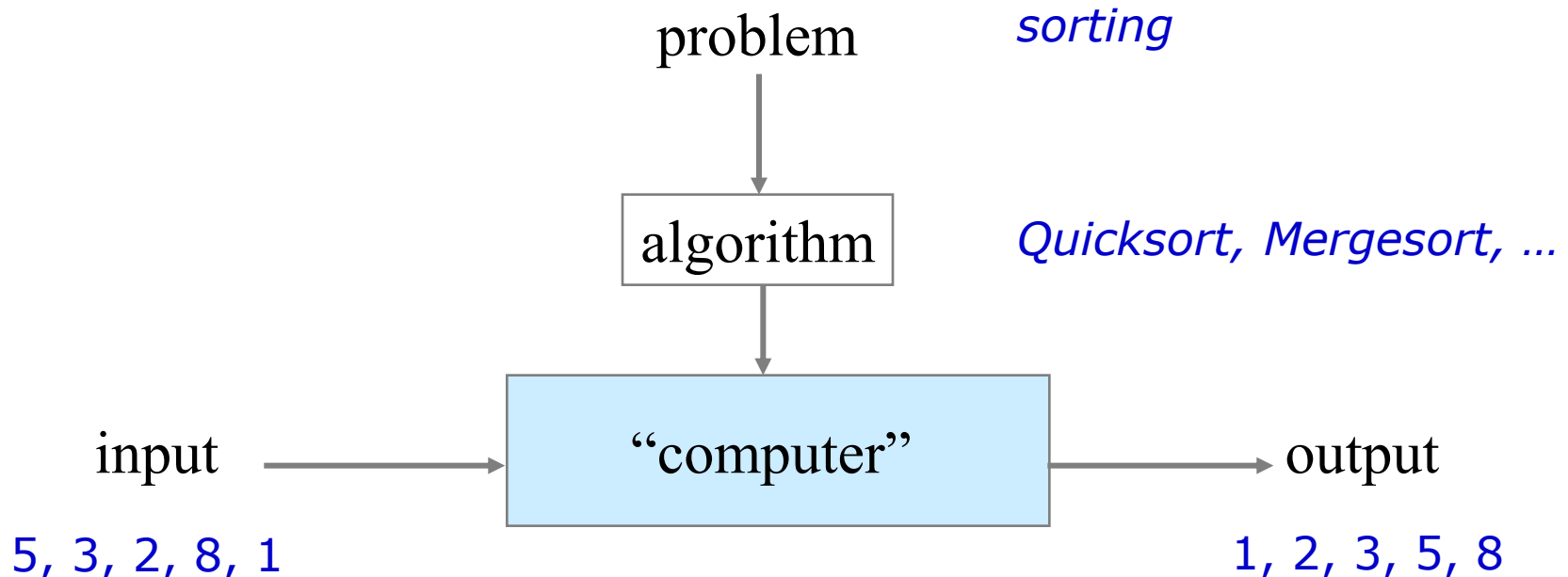
An ***algorithm*** is a sequence of *unambiguous* instructions for *solving a problem*, i.e., for obtaining a required *output* for any *legitimate input* in a *finite* amount of time.



Quick Review

□ What is an algorithm?

An ***algorithm*** is a sequence of *unambiguous* instructions for *solving a problem*, i.e., for obtaining a required *output* for any *legitimate input* in a *finite* amount of time.



Quick Review

□ Iterative algorithms vs. Recursive algorithms

```
function factorial (N: integer): integer;  
var i: integer;  
begin  
    factorial := 1;  
    for i:=2 to N do  
        factorial := factorial*i;  
end;
```

$$N! = 1*2*3*...*(N-1)*N$$

$$N! = N*(N-1)!$$

```
function factorial (N: integer): integer;  
begin  
    if N = 0  
    then factorial := 1  
    else factorial := N*factorial (N-1);  
end;
```




Quick Review

□ Iterative algorithms vs. Recursive algorithms

```
function factorial (N: integer): integer;  
var i: integer;  
begin  
    factorial := 1;  
    for i:=2 to N do  
        factorial := factorial*i;  
end;
```

$$N! = 1*2*3*...*(N-1)*N$$

$$N! = N*(N-1)!$$

Which one do you prefer?
WHY? Time, Memory Space?

```
function factorial (N: integer): integer;  
begin  
    if N = 0  
    then factorial := 1  
    else factorial := N*factorial (N-1);  
end;
```

factorial(10)

Recursion path

factorial(10)

10*factorial(9)

9*factorial(8)

8*factorial(7)

7*factorial(6)

6*factorial(5)

5*factorial(4)

4*factorial(3)

3*factorial(2)

2*factorial(1)

1*factorial(0)

Recursive cases
with smaller sizes

Base case

1*1 Return path → 1

3628800

10*9* 8*7*6*5*4*3*2*1*1

9* 8*7*6*5*4*3*2*1*1

8*7*6*5*4*3*2*1*1

7*6*5*4*3*2*1*1

6*5*4*3*2*1*1

5*4*3*2*1*1

4*3*2*1*1

3*2*1*1

2*1*1

1*1



Quick Review

- List the sequence of Fibonacci numbers

1, 1, 2, 3, 5, 8, 13, 21, 34, ...

```
function fibonacci_r (N: integer): integer;  
begin  
  if N <= 1  
  then fibonacci := 1  
  else fibonacci := fibonacci (N-1) + fibonacci (N-2);  
end;
```

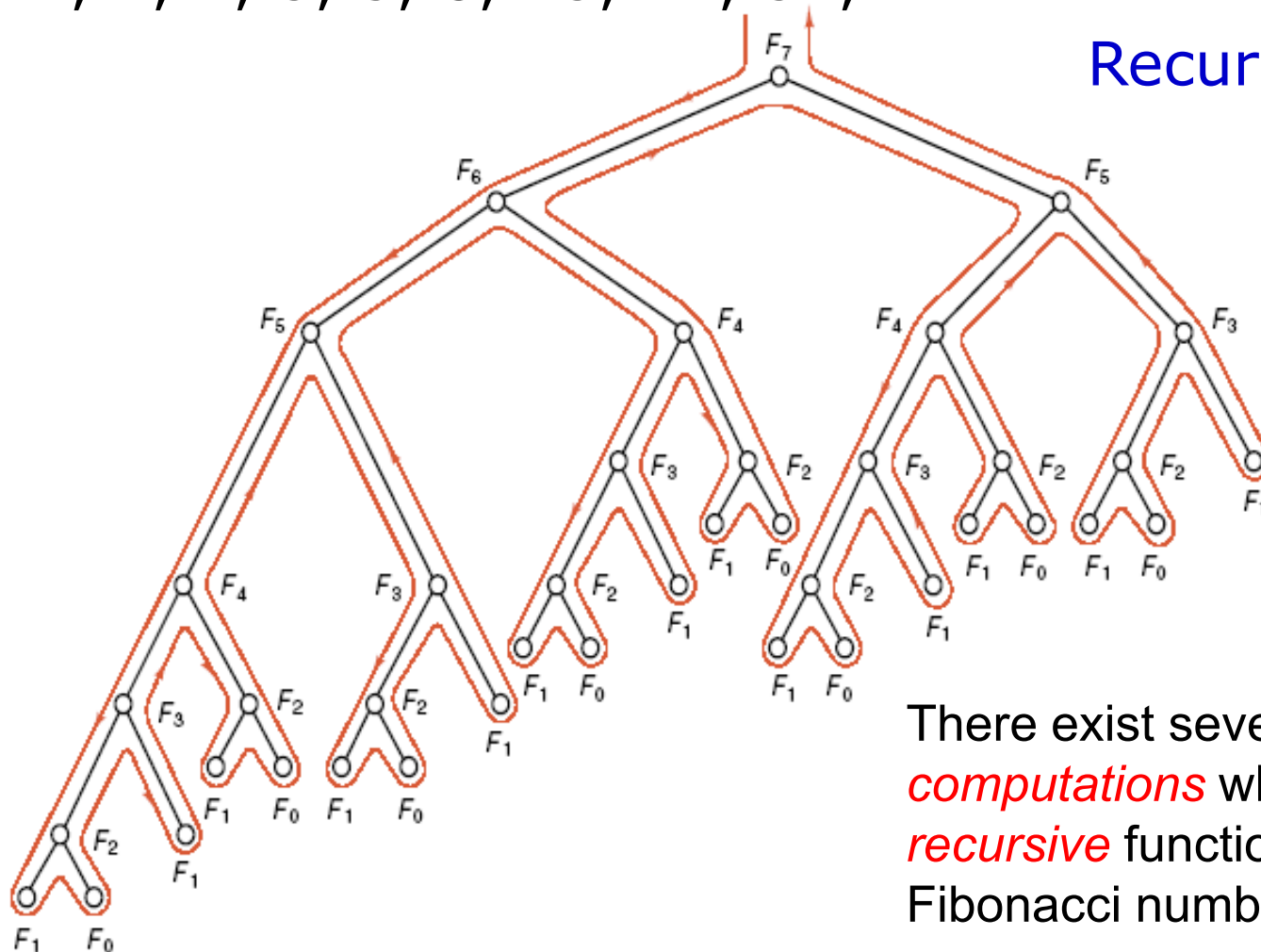
How about these functions?

```
function fibonacci_i (N: integer): integer;  
var      i: integer;  
        F: array [0..max] of integer; //max>=N  
begin  
  F[0] := 1; F[1] := 1;  
  for i := 2 to N do F[i] := F[i-1] + F[i-2];  
  fibonacci := F[N];  
end;
```

Quick Review

- List the sequence of Fibonacci numbers

1, 1, 2, 3, 5, 8, 13, 21, 34, ...



Recursive tree

There exist several *redundant computations* when using the *recursive* function to compute Fibonacci numbers!!!

Quick Review

□ Data structures

- Array
- Linked list
- Stack
- Queue
- Heap
- Tree
- Graph

□ Mathematics

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$

$$\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\sum_{i=0}^n 2^i = 2^{n+1} - 1$$

$$\sum_{i=0}^n a^i = \frac{a^{n+1}-1}{a-1} \text{ if } a > 1$$

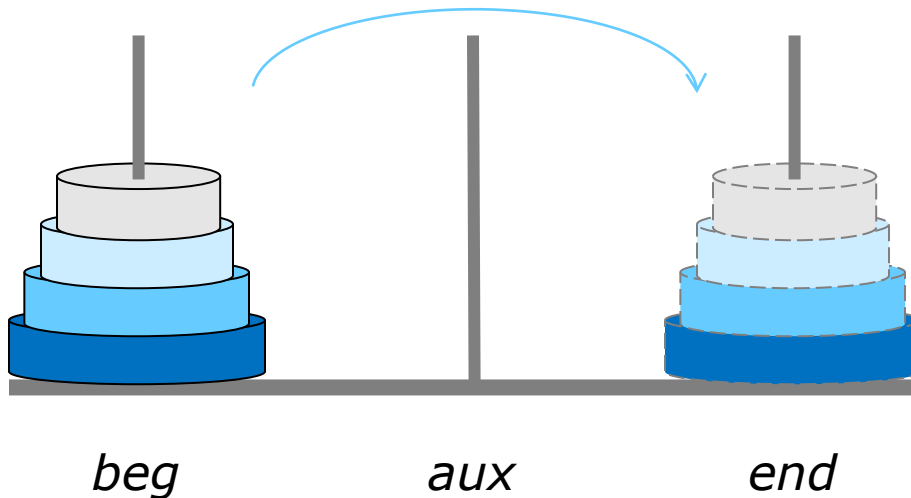
$$\sum_{i=0}^n a^i = \frac{1}{1-a} \text{ if } 0 < a < 1 \text{ and } n \rightarrow \infty$$

$$\sum_{i=1}^n cx_i = c \sum_{i=1}^n x_i$$

$$\sum_{i=1}^n (x_i + y_i) = \sum_{i=1}^n x_i + \sum_{i=1}^n y_i$$

1.1. Recursion and recurrence relations

Hanoi Tower



Move disks from *beg* to *end* using *aux* as a temporary holding area in such a way that smaller disks are always on top of larger disks and one disk is moved at a time.

Input:

n disks at *beg*

no disk at *aux* and *end*

Output:

n disks at *end*

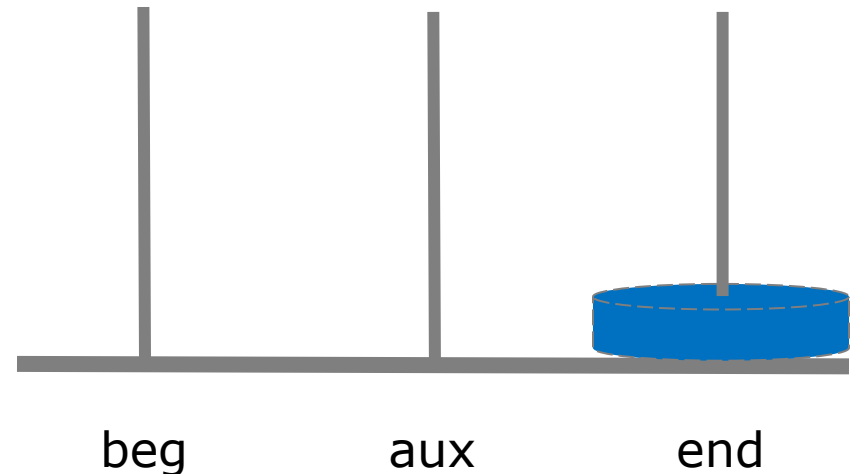
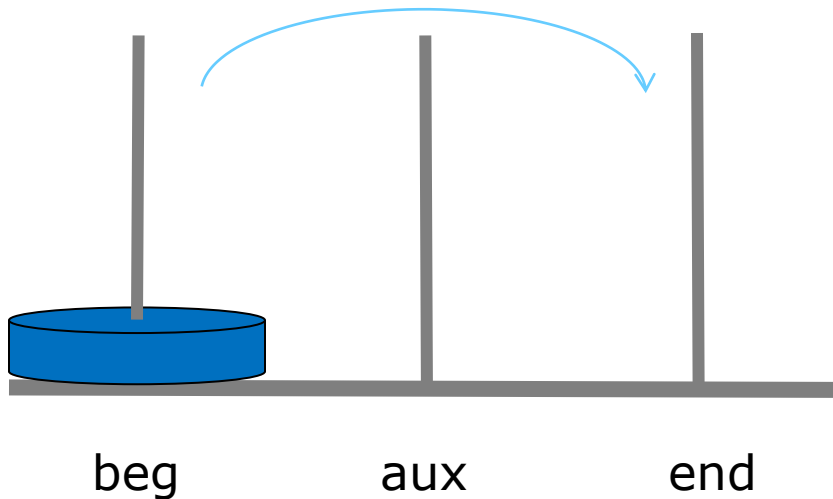
no disk at *beg* and *aux*

*How many moves are needed if *n* disks are given?*



1.1. Recursion and recurrence relations

Hanoi Tower



The number of moves:

$$n=1, \quad C_1 = 1$$

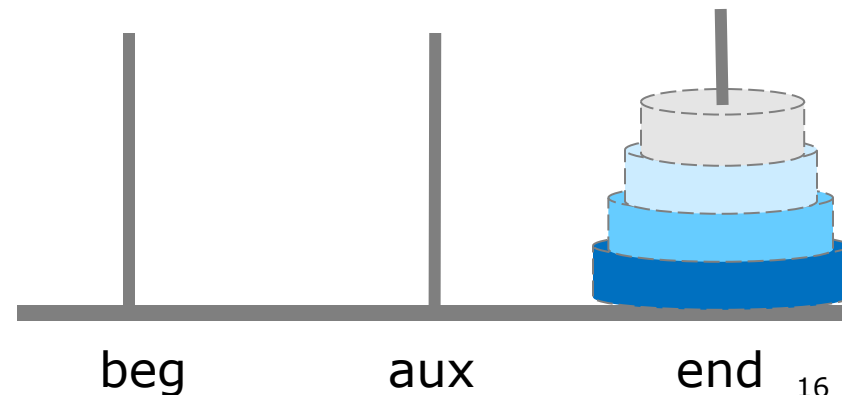
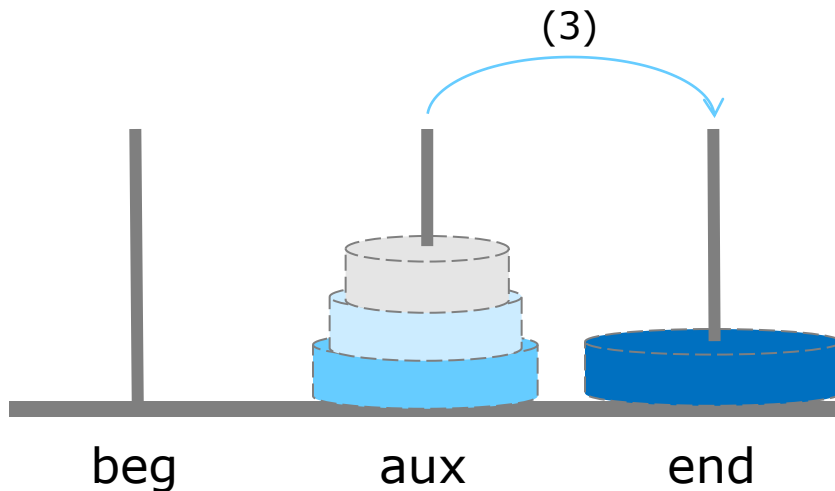
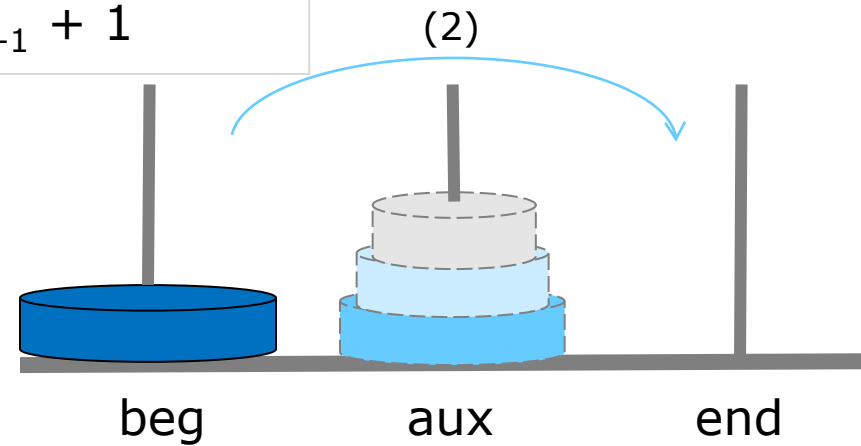
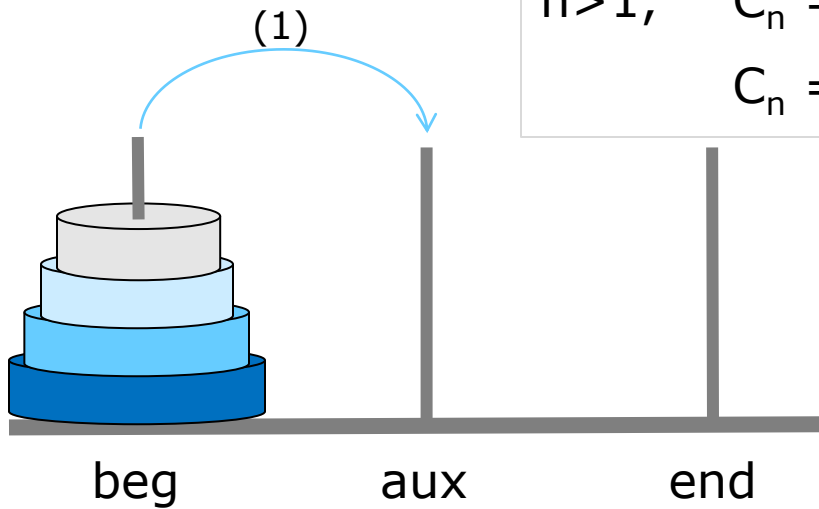
1.1. Recursion and recurrence relations

Hanoi Tower

The number of moves:

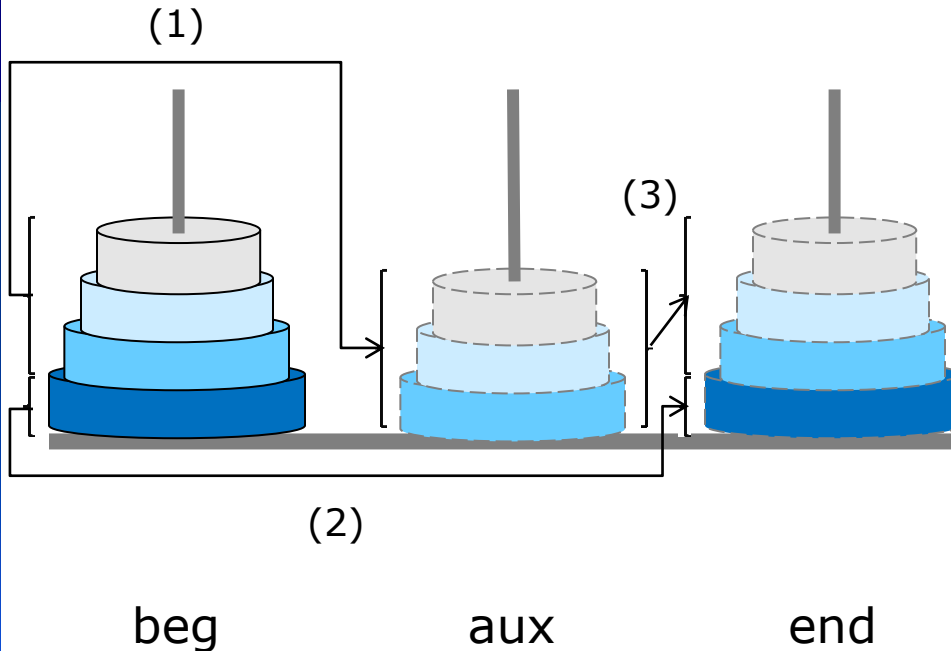
$$n > 1, \quad C_n = C_{n-1} + 1 + C_{n-1}$$

$$C_n = 2C_{n-1} + 1$$



1.1. Recursion and recurrence relations

Hanoi Tower



```

procedure hanoi(n, beg, aux, end);
begin
  if n = 1 then
    writeln(beg, end)
  else
    begin
      hanoi(n-1, beg, end, aux) ;
      writeln(beg, end);
      hanoi(n-1, aux, beg, end);
    end
  end;

```

The number of moves:

$$n=1, \quad C_1 = 1$$

$$n>1, \quad C_n = 2C_{n-1} + 1$$



1.1. Recursion and recurrence relations

□ Recurrence relation

- A function of integer parameters
- Derived as a complexity of a recursive algorithm

The number of moves made by a recursive algorithm for Hanoi Tower:

$$n=1, C_1 = 1$$

$$n>1, C_n = 2C_{n-1} + 1$$



1.1. Recursion and recurrence relations

□ Recurrence relation

- A function of integer parameters
- Derived as a complexity of a recursive algorithm
- Solved by substitution iteratively



1.1. Recursion and recurrence relations

□ Recurrence relation

- A function of integer parameters
- Derived as a complexity of a recursive algorithm
- Solved by substitution iteratively

→ *Further reading:*

Appendix B. Short Tutorial on Recurrence Relations

[2], pp. 473-485



1.1. Recursion and recurrence relations

- Solve recurrence relations by *substitution iteratively*

The number of moves in Hanoi Tower:

$$n=1, \quad C_1 = 1$$

$$n>1, \quad C_n = 2C_{n-1} + 1$$



1.1. Recursion and recurrence relations

- Solve recurrence relations by *substitution iteratively*

The number of moves in Hanoi Tower:

$$n=1, \quad C_1 = 1$$

$$n>1, \quad C_n = 2C_{n-1} + 1$$

$$C_n = 2(2C_{n-2} + 1) + 1$$



1.1. Recursion and recurrence relations

- Solve recurrence relations by *substitution iteratively*

The number of moves in Hanoi Tower:

$$n=1, \quad C_1 = 1$$

$$n>1, \quad C_n = 2C_{n-1} + 1$$

$$C_n = 2(2C_{n-2} + 1) + 1$$

$$C_n = 2^2C_{n-2} + 2 + 1$$



1.1. Recursion and recurrence relations

- Solve recurrence relations by *substitution iteratively*

The number of moves in Hanoi Tower:

$$n=1, \quad C_1 = 1$$

$$n>1, \quad C_n = 2C_{n-1} + 1$$

$$C_n = 2(2C_{n-2} + 1) + 1$$

$$C_n = 2^2C_{n-2} + 2 + 1$$

$$C_n = 2^2(2C_{n-3} + 1) + 2 + 1$$

$$C_n = 2^3C_{n-3} + 2^2 + 2 + 1$$

...

$$C_n = 2^{n-1}C_1 + 2^{n-2} + \dots + 2^2 + 2 + 1$$

$$C_n = 2^{n-1} + 2^{n-2} + \dots + 2^2 + 2 + 1$$

$$C_n = 2^n - 1$$



1.1. Recursion and recurrence relations

- Solve recurrence relations by *substitution iteratively*

$$n=1, \quad C_1 = 1$$

$$n>1, \quad C_n = 4C_{n/2} + n^3$$



1.1. Recursion and recurrence relations

- Solve recurrence relations by *substitution iteratively*

$$n=1, \quad C_1 = 1$$

$$n>1, \quad C_n = 4C_{n/2} + n^3$$

Supposed: $n = 2^k$



1.1. Recursion and recurrence relations

- Solve recurrence relations by *substitution iteratively*

$$n=1, \quad C_1 = 1$$

$$n>1, \quad C_n = 4C_{n/2} + n^3$$

Supposed: $n = 2^k$

C_n is rewritten as:

$$C_{2^k} = 4C_{2^{k-1}} + 2^{k^3}$$



1.1. Recursion and recurrence relations

- Solve recurrence relations by *substitution iteratively*

$$n=1, \quad C_1 = 1$$

$$n>1, \quad C_n = 4C_{n/2} + n^3$$

Supposed: $n = 2^k$

C_n is rewritten as:

$$C_{2^k} = 4C_{2^{k-1}} + 2^{k^3}$$

Simplified as:

$$\frac{C_{2^k}}{2^k} = \frac{4C_{2^{k-1}}}{2^k} + \frac{2^{k^3}}{2^k}$$



1.1. Recursion and recurrence relations

- Solve recurrence relations by *substitution iteratively*

$$n=1, \quad C_1 = 1$$

$$n>1, \quad C_n = 4C_{n/2} + n^3$$

Supposed: $n = 2^k$

C_n is rewritten as:

$$C_{2^k} = 4C_{2^{k-1}} + 2^{k^3}$$

Simplified as:

$$\frac{C_{2^k}}{2^k} = \frac{4C_{2^{k-1}}}{2^k} + \frac{2^{k^3}}{2^k}$$

$$\frac{C_{2^k}}{2^k} = \frac{2C_{2^{k-1}}}{2^{k-1}} + 2^k * 2^k$$



1.1. Recursion and recurrence relations

□ Solve recurrence relations by *substitution iteratively*

$$n=1, \quad C_1 = 1$$

$$n>1, \quad C_n = 4C_{n/2} + n^3$$

Supposed: $n = 2^k$

C_n is rewritten as:

$$C_{2^k} = 4C_{2^{k-1}} + 2^{k^3}$$

Simplified as:

$$\frac{C_{2^k}}{2^k} = \frac{4C_{2^{k-1}}}{2^k} + \frac{2^{k^3}}{2^k}$$

$$\frac{C_{2^k}}{2^k} = \frac{2C_{2^{k-1}}}{2^{k-1}} + 2^k * 2^k$$

Start substitution:

$$\frac{C_{2^k}}{2^k} = 2\left(\frac{2C_{2^{k-2}}}{2^{k-2}} + 2^{(k-1)} * 2^{(k-1)}\right) + 2^k * 2^k$$



1.1. Recursion and recurrence relations

□ Solve recurrence relations by *substitution iteratively*

$$n=1, \quad C_1 = 1$$

$$n>1, \quad C_n = 4C_{n/2} + n^3$$

Supposed: $n = 2^k$

C_n is rewritten as:

$$C_{2^k} = 4C_{2^{k-1}} + 2^{k^3}$$

Simplified as:

$$\frac{C_{2^k}}{2^k} = \frac{4C_{2^{k-1}}}{2^k} + \frac{2^{k^3}}{2^k}$$

$$\frac{C_{2^k}}{2^k} = \frac{2C_{2^{k-1}}}{2^{k-1}} + 2^k * 2^k$$

Start substitution:

$$\frac{C_{2^k}}{2^k} = 2 \left(\frac{2C_{2^{k-2}}}{2^{k-2}} + 2^{(k-1)} * 2^{(k-1)} \right) + 2^k * 2^k$$

$$\frac{C_{2^k}}{2^k} = \frac{2^2 C_{2^{k-2}}}{2^{k-2}} + 2^{k-1} * 2^k + 2^k * 2^k$$



1.1. Recursion and recurrence relations

□ Solve recurrence relations by *substitution iteratively*

$$n=1, \quad C_1 = 1$$

$$n>1, \quad C_n = 4C_{n/2} + n^3$$

Supposed: $n = 2^k$

C_n is rewritten as:

$$C_{2^k} = 4C_{2^{k-1}} + 2^{k^3}$$

Simplified as:

$$\frac{C_{2^k}}{2^k} = \frac{4C_{2^{k-1}}}{2^k} + \frac{2^{k^3}}{2^k}$$

$$\frac{C_{2^k}}{2^k} = \frac{2C_{2^{k-1}}}{2^{k-1}} + 2^k * 2^k$$

Start substitution:

$$\frac{C_{2^k}}{2^k} = 2 \left(\frac{2C_{2^{k-2}}}{2^{k-2}} + 2^{(k-1)} * 2^{(k-1)} \right) + 2^k * 2^k$$

$$\frac{C_{2^k}}{2^k} = \frac{2^2 C_{2^{k-2}}}{2^{k-2}} + 2^{k-1} * 2^k + 2^k * 2^k$$

$$\frac{C_{2^k}}{2^k} = 2^2 \left(\frac{2C_{2^{k-3}}}{2^{k-3}} + 2^{k-2} * 2^{k-2} \right) + 2^{k-1} * 2^k + 2^k * 2^k$$



1.1. Recursion and recurrence relations

□ Solve recurrence relations by *substitution iteratively*

$$n=1, \quad C_1 = 1$$

$$n>1, \quad C_n = 4C_{n/2} + n^3$$

Supposed: $n = 2^k$

C_n is rewritten as:

$$C_{2^k} = 4C_{2^{k-1}} + 2^{k^3}$$

Simplified as:

$$\frac{C_{2^k}}{2^k} = \frac{4C_{2^{k-1}}}{2^k} + \frac{2^{k^3}}{2^k}$$

$$\frac{C_{2^k}}{2^k} = \frac{2C_{2^{k-1}}}{2^{k-1}} + 2^k * 2^k$$

Start substitution:

$$\frac{C_{2^k}}{2^k} = 2 \left(\frac{2C_{2^{k-2}}}{2^{k-2}} + 2^{(k-1)} * 2^{(k-1)} \right) + 2^k * 2^k$$

$$\frac{C_{2^k}}{2^k} = \frac{2^2 C_{2^{k-2}}}{2^{k-2}} + 2^{k-1} * 2^k + 2^k * 2^k$$

$$\frac{C_{2^k}}{2^k} = 2^2 \left(\frac{2C_{2^{k-3}}}{2^{k-3}} + 2^{k-2} * 2^{k-2} \right) + 2^{k-1} * 2^k + 2^k * 2^k$$

$$\frac{C_{2^k}}{2^k} = \frac{2^3 C_{2^{k-3}}}{2^{k-3}} + 2^{k-2} * 2^k + 2^{k-1} * 2^k + 2^k * 2^k$$



1.1. Recursion and recurrence relations

□ Solve recurrence relations by *substitution iteratively*

$$n=1, \quad C_1 = 1$$

$$n>1, \quad C_n = 4C_{n/2} + n^3$$

Supposed: $n = 2^k$

C_n is rewritten as:

$$C_{2^k} = 4C_{2^{k-1}} + 2^{k^3}$$

Simplified as:

$$\frac{C_{2^k}}{2^k} = \frac{4C_{2^{k-1}}}{2^k} + \frac{2^{k^3}}{2^k}$$

$$\frac{C_{2^k}}{2^k} = \frac{2C_{2^{k-1}}}{2^{k-1}} + 2^k * 2^k$$

Start substitution:

$$\frac{C_{2^k}}{2^k} = 2 \left(\frac{2C_{2^{k-2}}}{2^{k-2}} + 2^{(k-1)} * 2^{(k-1)} \right) + 2^k * 2^k$$

$$\frac{C_{2^k}}{2^k} = \frac{2^2 C_{2^{k-2}}}{2^{k-2}} + 2^{k-1} * 2^k + 2^k * 2^k$$

$$\frac{C_{2^k}}{2^k} = 2^2 \left(\frac{2C_{2^{k-3}}}{2^{k-3}} + 2^{k-2} * 2^{k-2} \right) + 2^{k-1} * 2^k + 2^k * 2^k$$

$$\frac{C_{2^k}}{2^k} = \frac{2^3 C_{2^{k-3}}}{2^{k-3}} + 2^{k-2} * 2^k + 2^{k-1} * 2^k + 2^k * 2^k$$

...

$$\frac{C_{2^k}}{2^k} = \frac{2^k C_{2^{k-k}}}{2^{k-k}} + 2^{k-(k-1)} * 2^k + \dots + 2^{k-2} * 2^k + 2^{k-1} * 2^k + 2^k * 2^k$$



1.1. Recursion and recurrence relations

□ Solve recurrence relations by *substitution iteratively*

$$n=1, \quad C_1 = 1$$

$$n>1, \quad C_n = 4C_{n/2} + n^3$$

Supposed: $n = 2^k$

C_n is rewritten as:

$$C_{2^k} = 4C_{2^{k-1}} + 2^{k^3}$$

Simplified as:

$$\frac{C_{2^k}}{2^k} = \frac{4C_{2^{k-1}}}{2^k} + \frac{2^{k^3}}{2^k}$$

$$\frac{C_{2^k}}{2^k} = \frac{2C_{2^{k-1}}}{2^{k-1}} + 2^k * 2^k$$

Start substitution:

$$\frac{C_{2^k}}{2^k} = 2 \left(\frac{2C_{2^{k-2}}}{2^{k-2}} + 2^{(k-1)} * 2^{(k-1)} \right) + 2^k * 2^k$$

$$\frac{C_{2^k}}{2^k} = \frac{2^2 C_{2^{k-2}}}{2^{k-2}} + 2^{k-1} * 2^k + 2^k * 2^k$$

$$\frac{C_{2^k}}{2^k} = 2^2 \left(\frac{2C_{2^{k-3}}}{2^{k-3}} + 2^{k-2} * 2^{k-2} \right) + 2^{k-1} * 2^k + 2^k * 2^k$$

$$\frac{C_{2^k}}{2^k} = \frac{2^3 C_{2^{k-3}}}{2^{k-3}} + 2^{k-2} * 2^k + 2^{k-1} * 2^k + 2^k * 2^k$$

...

$$\frac{C_{2^k}}{2^k} = \frac{2^k C_{2^{k-k}}}{2^{k-k}} + 2^{k-(k-1)} * 2^k + \dots + 2^{k-2} * 2^k + 2^{k-1} * 2^k + 2^k * 2^k$$

$$\frac{C_{2^k}}{2^k} = \frac{2^k C_1}{2^0} + 2^1 * 2^k + \dots + 2^{k-2} * 2^k + 2^{k-1} * 2^k + 2^k * 2^k$$



1.1. Recursion and recurrence relations

□ Solve recurrence relations by *substitution iteratively*

$$n=1, \quad C_1 = 1$$

$$n>1, \quad C_n = 4C_{n/2} + n^3$$

Supposed: $n = 2^k$

C_n is rewritten as:

$$C_{2^k} = 4C_{2^{k-1}} + 2^{k^3}$$

Simplified as:

$$\frac{C_{2^k}}{2^k} = \frac{4C_{2^{k-1}}}{2^k} + \frac{2^{k^3}}{2^k}$$

$$\frac{C_{2^k}}{2^k} = \frac{2C_{2^{k-1}}}{2^{k-1}} + 2^k * 2^k$$

Start substitution:

$$\frac{C_{2^k}}{2^k} = 2 \left(\frac{2C_{2^{k-2}}}{2^{k-2}} + 2^{(k-1)} * 2^{(k-1)} \right) + 2^k * 2^k$$

$$\frac{C_{2^k}}{2^k} = \frac{2^2 C_{2^{k-2}}}{2^{k-2}} + 2^{k-1} * 2^k + 2^k * 2^k$$

...

$$\frac{C_{2^k}}{2^k} = \frac{2^k C_{2^{k-k}}}{2^{k-k}} + 2^{k-(k-1)} * 2^k + \dots + 2^{k-2} * 2^k + 2^{k-1} * 2^k + 2^k * 2^k$$

$$\frac{C_{2^k}}{2^k} = \frac{2^k C_1}{2^0} + 2^1 * 2^k + \dots + 2^{k-2} * 2^k + 2^{k-1} * 2^k + 2^k * 2^k$$

$$\frac{C_{2^k}}{2^k} = 2^k + 2^1 * 2^k + \dots + 2^{k-2} * 2^k + 2^{k-1} * 2^k + 2^k * 2^k$$



1.1. Recursion and recurrence relations

□ Solve recurrence relations by *substitution iteratively*

$$n=1, \quad C_1 = 1$$

$$n>1, \quad C_n = 4C_{n/2} + n^3$$

Supposed: $n = 2^k$

C_n is rewritten as:

$$C_{2^k} = 4C_{2^{k-1}} + 2^{k^3}$$

Simplified as:

$$\frac{C_{2^k}}{2^k} = \frac{4C_{2^{k-1}}}{2^k} + \frac{2^{k^3}}{2^k}$$

$$\frac{C_{2^k}}{2^k} = \frac{2C_{2^{k-1}}}{2^{k-1}} + 2^k * 2^k$$

Start substitution:

$$\frac{C_{2^k}}{2^k} = 2 \left(\frac{2C_{2^{k-2}}}{2^{k-2}} + 2^{(k-1)} * 2^{(k-1)} \right) + 2^k * 2^k$$

$$\frac{C_{2^k}}{2^k} = \frac{2^2 C_{2^{k-2}}}{2^{k-2}} + 2^{k-1} * 2^k + 2^k * 2^k$$

...

$$\frac{C_{2^k}}{2^k} = \frac{2^k C_{2^{k-k}}}{2^{k-k}} + 2^{k-(k-1)} * 2^k + \dots + 2^{k-2} * 2^k + 2^{k-1} * 2^k + 2^k * 2^k$$

$$\frac{C_{2^k}}{2^k} = \frac{2^k C_1}{2^0} + 2^1 * 2^k + \dots + 2^{k-2} * 2^k + 2^{k-1} * 2^k + 2^k * 2^k$$

$$\frac{C_{2^k}}{2^k} = 2^k + 2^1 * 2^k + \dots + 2^{k-2} * 2^k + 2^{k-1} * 2^k + 2^k * 2^k$$

$$\frac{C_{2^k}}{2^k} = 2^k (1 + 2^1 + \dots + 2^{k-2} + 2^{k-1} + 2^k)$$



1.1. Recursion and recurrence relations

□ Solve recurrence relations by *substitution iteratively*

$$n=1, \quad C_1 = 1$$

$$n>1, \quad C_n = 4C_{n/2} + n^3$$

Supposed: $n = 2^k$

C_n is rewritten as:

$$C_{2^k} = 4C_{2^{k-1}} + 2^{k^3}$$

Simplified as:

$$\frac{C_{2^k}}{2^k} = \frac{4C_{2^{k-1}}}{2^k} + \frac{2^{k^3}}{2^k}$$

$$\frac{C_{2^k}}{2^k} = \frac{2C_{2^{k-1}}}{2^{k-1}} + 2^k * 2^k$$

Start substitution:

$$\frac{C_{2^k}}{2^k} = 2 \left(\frac{2C_{2^{k-2}}}{2^{k-2}} + 2^{(k-1)} * 2^{(k-1)} \right) + 2^k * 2^k$$

$$\frac{C_{2^k}}{2^k} = \frac{2^2 C_{2^{k-2}}}{2^{k-2}} + 2^{k-1} * 2^k + 2^k * 2^k$$

...

$$\frac{C_{2^k}}{2^k} = \frac{2^k C_{2^{k-k}}}{2^{k-k}} + 2^{k-(k-1)} * 2^k + \dots + 2^{k-2} * 2^k + 2^{k-1} * 2^k + 2^k * 2^k$$

$$\frac{C_{2^k}}{2^k} = \frac{2^k C_1}{2^0} + 2^1 * 2^k + \dots + 2^{k-2} * 2^k + 2^{k-1} * 2^k + 2^k * 2^k$$

$$\frac{C_{2^k}}{2^k} = 2^k + 2^1 * 2^k + \dots + 2^{k-2} * 2^k + 2^{k-1} * 2^k + 2^k * 2^k$$

$$\frac{C_{2^k}}{2^k} = 2^k (1 + 2^1 + \dots + 2^{k-2} + 2^{k-1} + 2^k)$$

$$C_{2^k} = 2^k * 2^k (2^{k+1} - 1)$$



1.1. Recursion and recurrence relations

□ Solve recurrence relations by *substitution iteratively*

$$n=1, \quad C_1 = 1$$

$$n>1, \quad C_n = 4C_{n/2} + n^3$$

Supposed: $n = 2^k$

C_n is rewritten as:

$$C_{2^k} = 4C_{2^{k-1}} + 2^{k^3}$$

Simplified as:

$$\frac{C_{2^k}}{2^k} = \frac{4C_{2^{k-1}}}{2^k} + \frac{2^{k^3}}{2^k}$$

$$\frac{C_{2^k}}{2^k} = \frac{2C_{2^{k-1}}}{2^{k-1}} + 2^k * 2^k$$

Start substitution:

$$\frac{C_{2^k}}{2^k} = 2 \left(\frac{2C_{2^{k-2}}}{2^{k-2}} + 2^{(k-1)} * 2^{(k-1)} \right) + 2^k * 2^k$$

$$\frac{C_{2^k}}{2^k} = \frac{2^2 C_{2^{k-2}}}{2^{k-2}} + 2^{k-1} * 2^k + 2^k * 2^k$$

...

$$\frac{C_{2^k}}{2^k} = \frac{2^k C_{2^{k-k}}}{2^{k-k}} + 2^{k-(k-1)} * 2^k + \dots + 2^{k-2} * 2^k + 2^{k-1} * 2^k + 2^k * 2^k$$

$$\frac{C_{2^k}}{2^k} = \frac{2^k C_1}{2^0} + 2^1 * 2^k + \dots + 2^{k-2} * 2^k + 2^{k-1} * 2^k + 2^k * 2^k$$

$$\frac{C_{2^k}}{2^k} = 2^k + 2^1 * 2^k + \dots + 2^{k-2} * 2^k + 2^{k-1} * 2^k + 2^k * 2^k$$

$$\frac{C_{2^k}}{2^k} = 2^k (1 + 2^1 + \dots + 2^{k-2} + 2^{k-1} + 2^k)$$

$$C_{2^k} = 2^k * 2^k (2^{k+1} - 1) \rightarrow C_n = 2n^3 - n^2$$



1.1. Recursion and recurrence relations

Your turn:

□ Solve recurrence relations by *substitution iteratively*

$$(1.1.1) \quad \begin{array}{l} C_n = c + C_{n-1} \quad \text{when } n > 1 \\ C_1 = d \end{array}$$

$$(1.1.2) \quad \begin{array}{l} C_n = 2C_{n/2} + n^2 \quad \text{when } n \geq 2 \\ C_1 = 1 \end{array}$$

$$(1.1.3) \quad \begin{array}{l} C_n = 2C_{n/3} + 1 \quad \text{when } n \geq 2 \\ C_1 = 1 \end{array}$$



1.1. Recursion and recurrence relations

Your turn:

□ Solve recurrence relations by *substitution iteratively*

$$(1.1.1) \quad \begin{aligned} C_n &= c + C_{n-1} && \text{when } n > 1 \\ C_1 &= d \end{aligned}$$

$$C_n = (n-1)c + d = cn - c + d$$

$$(1.1.2) \quad \begin{aligned} C_n &= 2C_{n/2} + n^2 && \text{when } n \geq 2 \\ C_1 &= 1 \end{aligned}$$

$$C_n = n(2n - 1) = 2n^2 - n$$

$$(1.1.3) \quad \begin{aligned} C_n &= 2C_{n/3} + 1 && \text{when } n \geq 2 \\ C_1 &= 1 \end{aligned}$$

$$C_n = 2n^{\log_3 2} - 1$$



1.2. Analysis of algorithms

- Analysis of an algorithm: *estimate* the resources used by that algorithm.
- Resources: Memory space
 Computational time



1.2. Analysis of algorithms

- Analysis of an algorithm: *estimate* the resources used by that algorithm.
- Resources: Memory space
 Computational time



1.2. Analysis of algorithms

- ❑ Step 1: Characterize the data which is to be used as input to the algorithm and to decide what type of analysis is appropriate.
 - Worst, Average, Best cases
- ❑ Step 2: identify *abstract operation* upon which the algorithm is based.
 - The number of abstract operations depends on a few quantities.
- ❑ Step 3: Proceed to the mathematical analysis to find the values for each of the fundamental quantities.

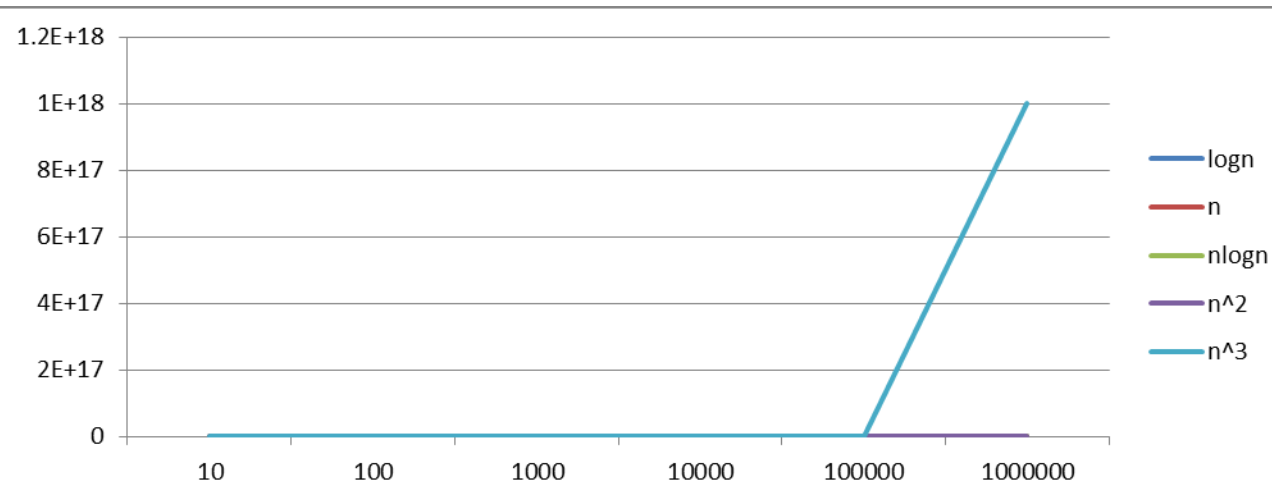
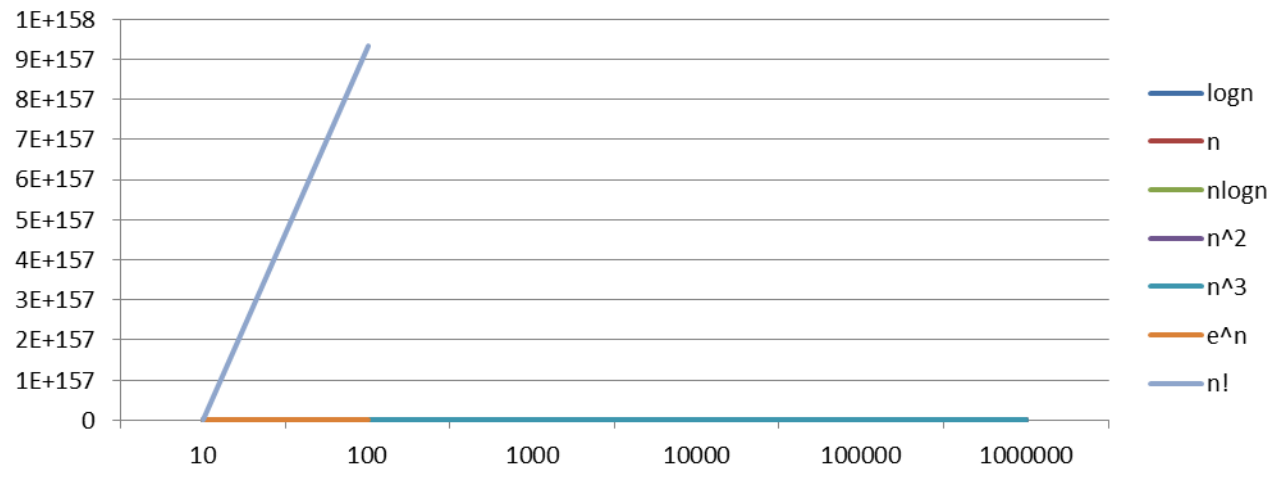


1.2. Analysis of algorithms

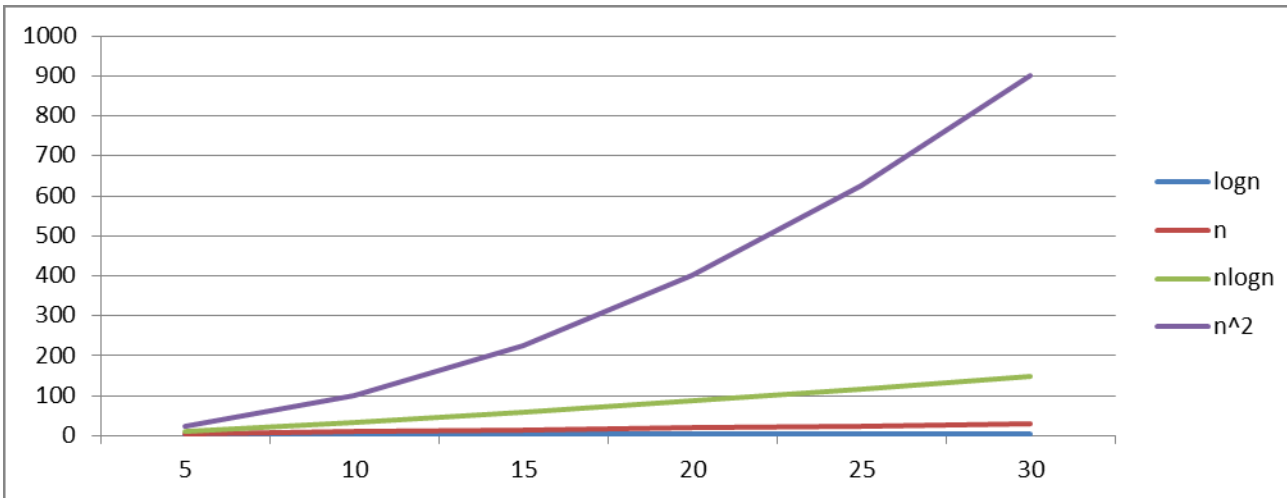
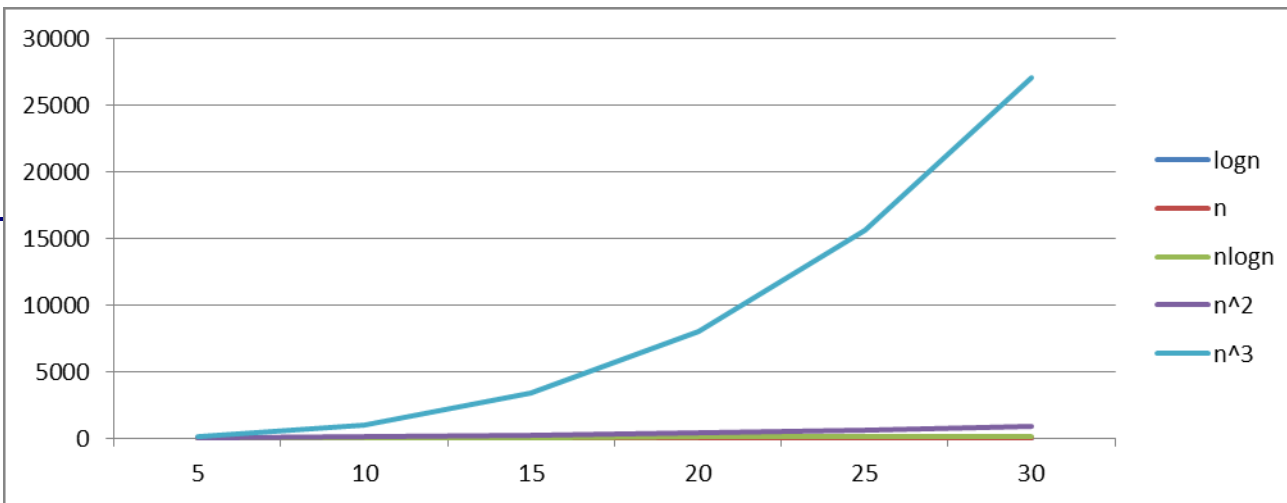
□ Efficiency classes

Class	Name	Comments
1	constant	Abstract operation is done a few times.
$\log n$	logarithmic	A logarithmic algorithm does not take all the input into account.
n	linear	Abstract operation is done whenever the algorithm scans the input of size n one by one.
$n \log n$	$n \cdot \log n$	Divide-and-conquer algorithms with division by 2: Quicksort, Mergesort in the average case
n^2	quadratic	Algorithms with two embedded loops. Abstract operation is done in the most inner loop.
n^3	cubic	Algorithms with three embedded loops
2^n	exponential	Typical for algorithms that generate all subsets of an n -element set
$n!$	factorial	Typical for algorithms that generate all permutations of an n -element set.

n : size of the input, a typical parameter $\rightarrow$ array of n elements, graph of n nodes 45



n	$\log n$	n	$n \log n$	n^2	n^3	e^n	$n!$
10	3.321928	10	33.21928	100	1000	22026.47	3628800
100	6.643856	100	664.3856	10000	1000000	2.69E+43	9.3E+157
1000	9.965784	1000	9965.784	1000000	1E+09		
10000	13.28771	10000	132877.1	1E+08	1E+12		
100000	16.60964	100000	1660964	1E+10	1E+15		
1000000	19.93157	1000000	19931569	1E+12	1E+18		

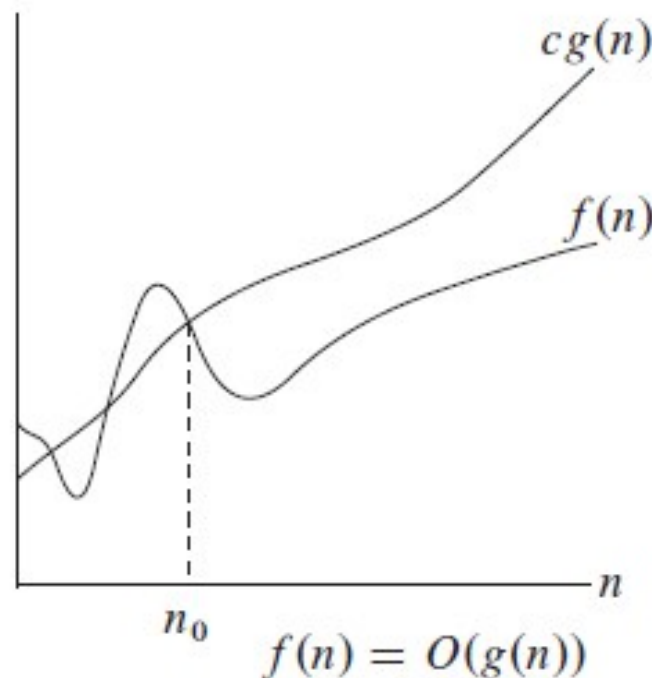


n	logn	n	nlogn	n^2	n^3	e^n	n!
5	2.321928	5	11.60964	25	125	148.4132	120
10	3.321928	10	33.21928	100	1000	22026.47	3628800
15	3.906891	15	58.60336	225	3375		
20	4.321928	20	86.43856	400	8000		
25	4.643856	25	116.0964	625	15625		
30	4.906891	30	147.2067	900	27000		

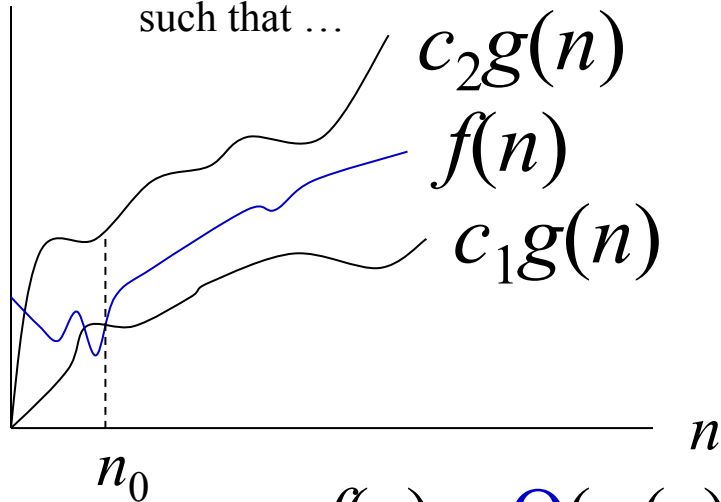
1.2. Analysis of algorithms

Big O's definition: A function $f(n)$ is said to be $O(g(n))$ if there exist positive constants c and n_0 such that $f(n)$ is less than or equal to $cg(n)$ for all $n \geq n_0$.

$$f(n) = O(g(n)) \quad \text{if} \quad f(n) \leq cg(n), \forall n \geq n_0, n_0 > 0, c > 0$$

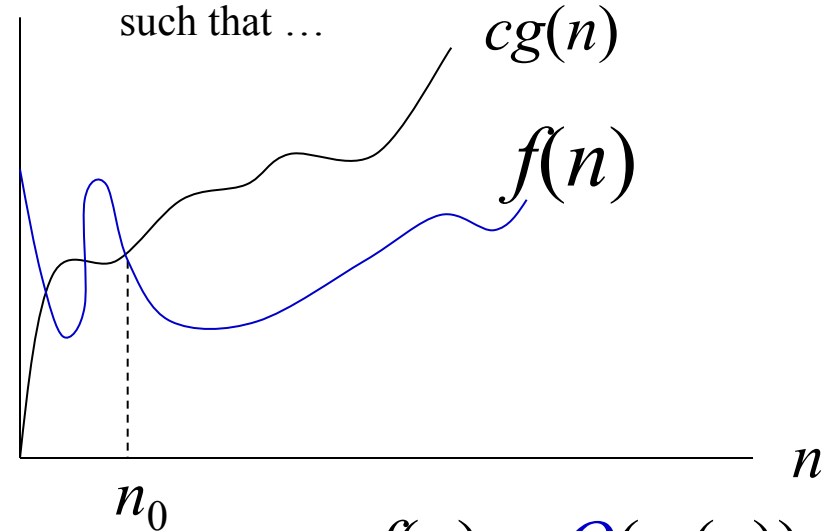


There exist positive constants c_1 and c_2 such that there is a positive constant n_0 such that ...



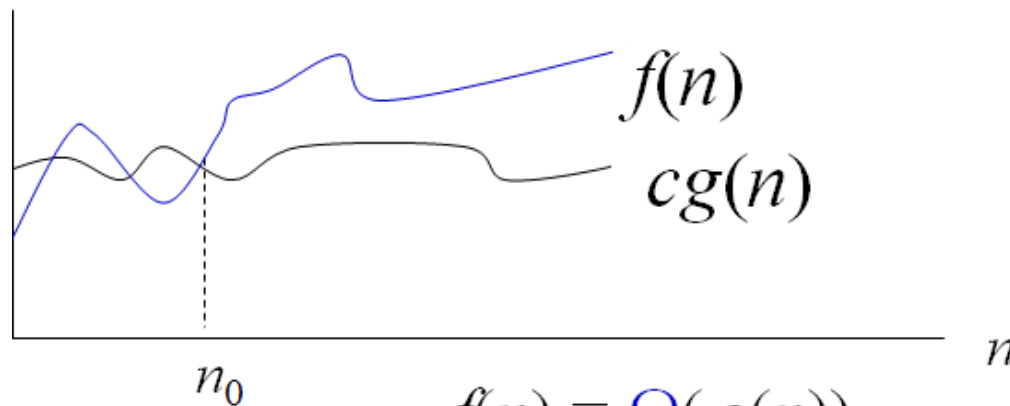
$$f(n) = \Theta(g(n))$$

There exists a positive constant c such that there is a positive constant n_0 such that ...



$$f(n) = O(g(n))$$

There exists a positive constant c such that there is a positive constant n_0 such that ...



$$f(n) = \Omega(g(n))$$

THEOREM If $t_1(n) \in O(g_1(n))$ and $t_2(n) \in O(g_2(n))$, then

$$t_1(n) + t_2(n) \in O(\max\{g_1(n), g_2(n)\}).$$

(The analogous assertions are true for the Ω and Θ notations as well.)

Using limits for comparing orders of growth:

$$\lim_{n \rightarrow \infty} \frac{t(n)}{g(n)} = \begin{cases} 0 & \text{implies that } t(n) \text{ has a smaller order of growth than } g(n) \\ c > 0 & \text{implies that } t(n) \text{ has the same order of growth as } g(n) \\ \infty & \text{implies that } t(n) \text{ has a larger order of growth than } g(n). \end{cases}$$

Note that the first two cases mean that $t(n) \in O(g(n))$, the last two mean that $t(n) \in \Omega(g(n))$, and the second case means that $t(n) \in \Theta(g(n))$.

L'Hôpital's rule:

$$\lim_{n \rightarrow \infty} \frac{t(n)}{g(n)} = \lim_{n \rightarrow \infty} \frac{t'(n)}{g'(n)}$$

[2] Levitin, A., *Introduction to the Design and Analysis of Algorithms*, Addison Wesley, 2003, pp. 57.



1.2. Analysis of algorithms

1.1. Identify the upper bound of the time complexity of algorithm A which is: $f(n) = 3n^2/4 + 1$.

a. $3/4$

b. n

c. $3n + 1$

d. n^2

$$f(n) = 3n^2/4 + 1 = O(n^2)$$



1.2. Analysis of algorithms

1.1. Identify the upper bound of the time complexity of algorithm A which is: $f(n) = 3n^2/4 + 1$.

a. $3/4$

b. n

c. $3n + 1$

d. n^2

$$f(n) = 3n^2/4 + 1 = O(n^2)$$

Because: choose: $c = 1, n_0 = 2, g(n) = n^2$

$$3n^2/4 + 1 \leq n^2 \Leftrightarrow 4 \leq n^2 \text{ always true } \forall n \geq n_0$$

$$\Leftrightarrow f(n) \leq cg(n), \forall n \geq n_0 \rightarrow f(n) = O(g(n)) = O(n^2)$$



1.2. Analysis of algorithms

1.1. Identify the upper bound of the time complexity of algorithm A which is: $f(n) = 3n^2/4 + 1$.

a. $3/4$

b. n

c. $3n + 1$

d. n^2

$$f(n) = 3n^2/4 + 1 = O(n^2)$$

Because:

$$\lim_{n \rightarrow \infty} \frac{\frac{3n^2}{4} + 1}{n^2} = \lim_{n \rightarrow \infty} \frac{3n/2}{2n} = \frac{3}{4} \rightarrow f(n) = \Theta(n^2) \rightarrow f(n) = O(n^2)$$



1.3. Analysis of iterative algorithms

- Given an iterative algorithm that finds the largest element in an array of n elements

```
procedure MAX(A, n, max)
/* Set max to the maximum of A(1:n) */
begin
    integer i, n;
    max := A[1];
    for i:= 2 to n do
        if A[i] > max then max := A[i]
    end
```



1.3. Analysis of iterative algorithms

□ Step 1:

- Characterize the data: one input parameter n
- Type of analysis: *all cases* (worst, average, best)

□ Step 2:

- Abstract operation: *comparison*

□ Step 3:

- *How many times is comparison* done when the algorithm runs on the input of n elements?

→ Your answer is ...



1.3. Analysis of iterative algorithms

- Given an iterative algorithm that finds the largest element in an array of n elements

```
procedure MAX(A, n, max)
/* Set max to the maximum of A(1:n) */
begin
  integer i, n;
  max := A[1];
  for i:= 2 to n do
    if A[i] > max then max := A[i]
end
```

The number of *comparisons* in all cases
= the number of iterations of "for" loop
= $n-1$



1.3. Analysis of iterative algorithms

- Given an iterative algorithm that finds the largest element in an array of n elements

```
procedure MAX(A, n, max)
/* Set max to the maximum of A(1:n) */
begin
  integer i, n;
  max := A[1];
  for i:= 2 to n do
    if A[i] > max then max := A[i]
end
```

The number of *comparisons* in all cases
= the number of iterations of "for" loop
= $n-1 \rightarrow f(n) = n-1 = O(n)$



1.3. Analysis of iterative algorithms

- Given the following program, what is its complexity in the case *when all the elements are unique or when the two last elements are the same*? It is supposed that *comparison* is chosen as abstract operation.

```
function UniqueElements (A, n)
begin
  for i:= 1 to n-1 do
    for j:= i + 1 to n do
      if A[i] = A[j] return false
  return true
end
```

2	3	4	0	8	1	true
5	2	1	3	7	7	false
1	2	0	9	1	3	false
8	4	7	6	4	5	false

How many *comparisons* need to be done?



1.3. Analysis of iterative algorithms

- Given the following program, what is its complexity in the case *when all the elements are unique or when the two last elements are the same*? It is supposed that *comparison* is chosen as abstract operation.

```
function UniqueElements (A, n)
begin
  for i:= 1 to n-1 do
    for j:= i + 1 to n do
      if A[i] = A[j] return false
    return true
end
```

- Step 1: one parameter n , *analysis case* given (*worst, average, best?*)
- Step 2: abstract operation = *comparison*
- Step 3: *how many comparisons* have been done in this case?



1.3. Analysis of iterative algorithms

- Given the following program, what is its complexity in the case *when all the elements are unique or when the two last elements are the same*? It is supposed that *comparison* is chosen as abstract operation.

```
function UniqueElements (A, n)
begin
  for i:= 1 to n-1 do
    for j:= i + 1 to n do
      if A[i] = A[j] return false
    return true
end
```

- Step 3: *how many cmps?*

$i = 1, j = 2..n$: $n-1$ cmps

$i = 2, j = 3..n$: $n-2$ cmps

...

$i = n-2, j = n-1..n$: 2 cmps

$i = n-1, j = n..n$: 1 cmps

→ $(n-1) + (n-2) + .. + 2 + 1$
 $= n(n-1)/2$ cmps

→ $C_n = n(n-1)/2 = O(n^2)$



1.3. Analysis of iterative algorithms

- Given the following program, what is its complexity in the case *when all the elements are unique or when the two last elements are the same*? It is supposed that *comparison* is chosen as abstract operation.

```
function UniqueElements (A, n)
begin
  for i:= 1 to n-1 do
    for j:= i + 1 to n do
      if A[i] = A[j] return false
    return true
end
```

- Step 1: one parameter n , *analysis case* given (*worst, average, best?*)
- Step 2: abstract operation = *comparison*
- Step 3: *how many comparisons* in this case?

$$C_n = n(n-1)/2 = O(n^2)$$



1.3. Analysis of iterative algorithms

Your turn:

□ Algorithm for what problem = ?

Abstract operation = ? Complexity class = ?

ALGORITHM $GE(A[0..n-1, 0..n])$

//Input: An n -by- $n+1$ matrix $A[0..n-1, 0..n]$ of real numbers

for $i \leftarrow 0$ **to** $n-2$ **do**

for $j \leftarrow i+1$ **to** $n-1$ **do**

for $k \leftarrow i$ **to** n **do**

$A[j, k] \leftarrow A[j, k] - A[i, k] * A[j, i] / A[i, i]$



1.3. Analysis of iterative algorithms

Your turn:

❑ Algorithm for what problem = ?

Abstract operation = ? Complexity class = ?

ALGORITHM $GE(A[0..n-1, 0..n])$

//Input: An n -by- $n+1$ matrix $A[0..n-1, 0..n]$ of real numbers

for $i \leftarrow 0$ **to** $n-2$ **do**

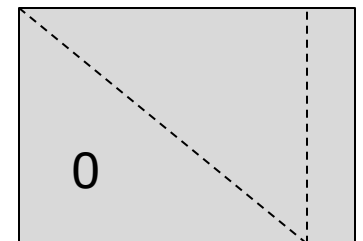
for $j \leftarrow i+1$ **to** $n-1$ **do**

for $k \leftarrow i$ **to** n **do**

$A[j, k] \leftarrow A[j, k] - A[i, k] * A[j, i] / A[i, i]$

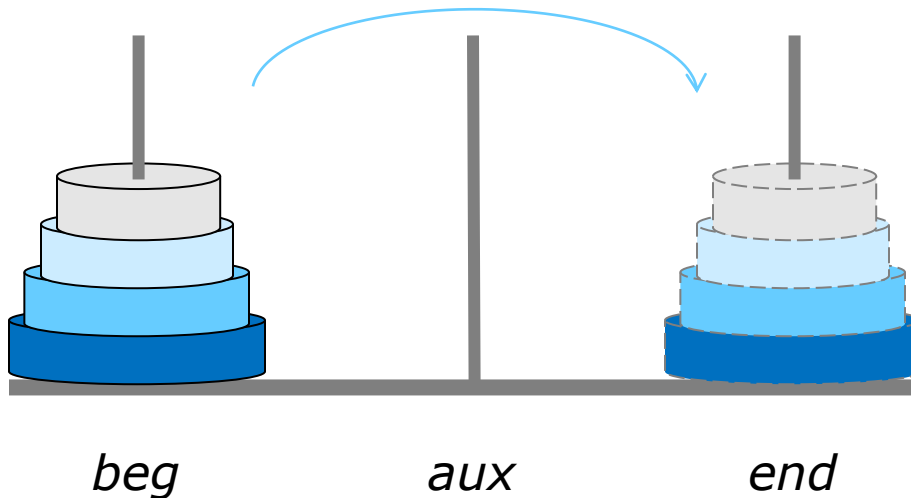
Gaussian Elimination? Multiplication? $(n-1)n(2n+5)/6?$

$O(n^3)?$



1.4. Analysis of recursive algorithms

Hanoi Tower

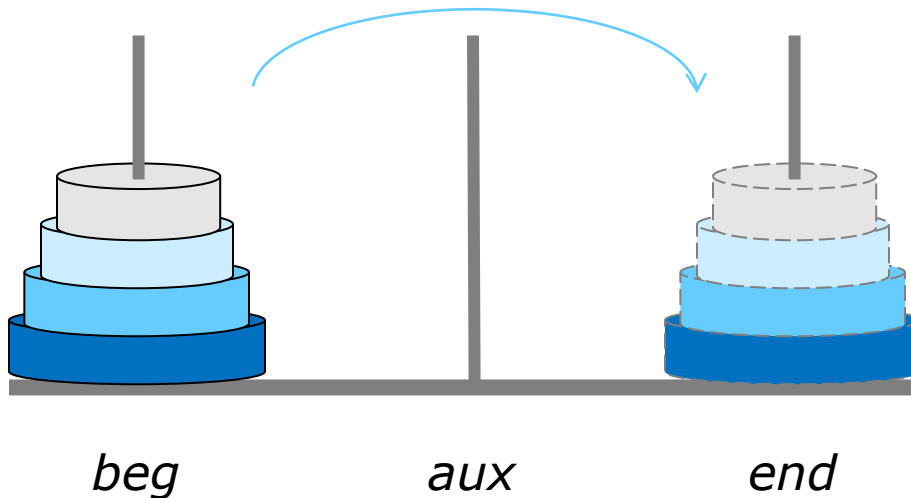


```
procedure hanoi(n, beg, aux, end);  
begin  
  if n = 1 then  
    writeln(beg, end)  
  else  
    begin  
      hanoi(n-1, beg, end, aux) ;  
      writeln(beg, end);  
      hanoi(n-1, aux, beg, end);  
    end  
  end;
```

- Step 1: one input parameter n , all analysis cases (worst, average, best)
- Step 2: abstract operation = *move*
- Step 3: *how many moves* are done when the algorithm runs on n disks?

1.4. Analysis of recursive algorithms

Hanoi Tower



```

procedure hanoi(n, beg, aux, end);
begin
  if n = 1 then
    writeln(beg, end)
  else
    begin
      hanoi(n-1, beg, end, aux) ;
      writeln(beg, end);
      hanoi(n-1, aux, beg, end);
    end
  end;

```

The number of moves:

$$n=1, \quad C_1 = 1$$

$$n>1, \quad C_n = 2C_{n-1} + 1$$

What is a more
particular number?



1.4. Analysis of recursive algorithms

- Solve recurrence relations by *substitution iteratively*

The number of moves in Hanoi Tower:

$$n=1, \quad C_1 = 1$$

$$n>1, \quad C_n = 2C_{n-1} + 1$$

$$C_n = 2(2C_{n-2} + 1) + 1$$

$$C_n = 2^2C_{n-2} + 2 + 1$$

$$C_n = 2^2(2C_{n-3} + 1) + 2 + 1$$

$$C_n = 2^3C_{n-3} + 2^2 + 2 + 1$$

...

$$C_n = 2^{n-1}C_1 + 2^{n-2} + \dots + 2^2 + 2 + 1$$

$$C_n = 2^{n-1} + 2^{n-2} + \dots + 2^2 + 2 + 1$$

$$C_n = 2^n - 1 = O(2^n)$$



1.4. Analysis of recursive algorithms

- Given a following recursive algorithm, what is its complexity? Supposed that *multiplication* is abstract operation.

```
algorithm S(n)
// input: a positive integer n
begin
    if n = 1 then return 1;
    else return S(n-1) + n*n*n;
end
```

return →
$$S(n) = 1 + 2^3 + 3^3 + \dots + (n-1)^3 + n^3$$



1.4. Analysis of recursive algorithms

- Given a following recursive algorithm, what is its complexity? Supposed that *multiplication* is abstract operation.

algorithm S(n)

// input: a positive integer n

begin

if n = 1 **then** return 1;

else return S(n-1) + n*n*n;

end

Base case:

no multiplication

→ $C_1 = 0$

return →

$$S(n) = 1 + 2^3 + 3^3 + \dots + (n-1)^3 + n^3$$



1.4. Analysis of recursive algorithms

- Given a following recursive algorithm, what is its complexity? Supposed that *multiplication* is abstract operation.

```
algorithm S(n)
// input: a positive integer n
begin
    if n = 1 then return 1;
    else return S(n-1) + n*n*n;
end
```

Recursive case:

multiplications in S(n-1)
and $n*n*n$

$\rightarrow C_n = C_{n-1} + 2$ when $n > 1$

return $\rightarrow$

$$S(n) = 1 + 2^3 + 3^3 + \dots + (n-1)^3 + n^3$$



1.4. Analysis of recursive algorithms

- Given a following recursive algorithm, what is its complexity? Supposed that *multiplication* is abstract operation.

```
algorithm S(n)
// input: a positive integer n
begin
    if n = 1 then return 1;
    else return S(n-1) + n*n*n;
end
```

recurrence
relation



$$\begin{aligned} C_1 &= 0, \\ C_n &= C_{n-1} + 2 \text{ when } n > 1 \end{aligned}$$



1.4. Analysis of recursive algorithms

- Solve recurrence relations by *substitution iteratively*

$$n=1, \quad C_1 = 0$$

$$n>1, \quad C_n = C_{n-1} + 2$$



1.4. Analysis of recursive algorithms

- Solve recurrence relations by *substitution iteratively*

$$n=1, \quad C_1 = 0$$

$$n>1, \quad C_n = C_{n-1} + 2$$

$$C_n = (C_{n-2} + 2) + 2$$



1.4. Analysis of recursive algorithms

- Solve recurrence relations by *substitution iteratively*

$$n=1, \quad C_1 = 0$$

$$n>1, \quad C_n = C_{n-1} + 2$$

$$C_n = (C_{n-2} + 2) + 2$$

$$C_n = C_{n-2} + 2*2$$



1.4. Analysis of recursive algorithms

- Solve recurrence relations by *substitution iteratively*

$$n=1, \quad C_1 = 0$$

$$n>1, \quad C_n = C_{n-1} + 2$$

$$C_n = (C_{n-2} + 2) + 2$$

$$C_n = C_{n-2} + 2*2$$

$$C_n = (C_{n-3} + 2) + 2*2$$



1.4. Analysis of recursive algorithms

- Solve recurrence relations by *substitution iteratively*

$$n=1, \quad C_1 = 0$$

$$n>1, \quad C_n = C_{n-1} + 2$$

$$C_n = (C_{n-2} + 2) + 2$$

$$C_n = C_{n-2} + 2*2$$

$$C_n = (C_{n-3} + 2) + 2*2$$

$$C_n = C_{n-3} + 3*2$$



1.4. Analysis of recursive algorithms

- Solve recurrence relations by *substitution iteratively*

$$n=1, \quad C_1 = 0$$

$$n>1, \quad C_n = C_{n-1} + 2$$

$$C_n = (C_{n-2} + 2) + 2$$

$$C_n = C_{n-2} + 2*2$$

$$C_n = (C_{n-3} + 2) + 2*2$$

$$C_n = C_{n-3} + 3*2$$

...

$$C_n = C_{n-(n-1)} + (n-1)*2$$



1.4. Analysis of recursive algorithms

- Solve recurrence relations by *substitution iteratively*

$$n=1, \quad C_1 = 0$$

$$n>1, \quad C_n = C_{n-1} + 2$$

$$C_n = (C_{n-2} + 2) + 2$$

$$C_n = C_{n-2} + 2*2$$

$$C_n = (C_{n-3} + 2) + 2*2$$

$$C_n = C_{n-3} + 3*2$$

...

$$C_n = C_{n-(n-1)} + (n-1)*2$$

$$C_n = C_1 + (n-1)*2$$



1.4. Analysis of recursive algorithms

- Solve recurrence relations by *substitution iteratively*

$$n=1, \quad C_1 = 0$$

$$n>1, \quad C_n = C_{n-1} + 2$$

$$C_n = (C_{n-2} + 2) + 2$$

$$C_n = C_{n-2} + 2*2$$

$$C_n = (C_{n-3} + 2) + 2*2$$

$$C_n = C_{n-3} + 3*2$$

...

$$C_n = C_{n-(n-1)} + (n-1)*2$$

$$C_n = C_1 + (n-1)*2$$

$$C_n = 2*(n-1)$$



1.4. Analysis of recursive algorithms

- Solve recurrence relations by *substitution iteratively*

$$n=1, \quad C_1 = 0$$

$$n>1, \quad C_n = C_{n-1} + 2$$

$$C_n = (C_{n-2} + 2) + 2$$

$$C_n = C_{n-2} + 2*2$$

$$C_n = (C_{n-3} + 2) + 2*2$$

$$C_n = C_{n-3} + 3*2$$

...

$$C_n = C_{n-(n-1)} + (n-1)*2$$

$$C_n = C_1 + (n-1)*2$$

$$C_n = 2*(n-1) = O(n)$$



1.4. Analysis of recursive algorithms

Your turn:

□ Abstract operation = ? Complexity = ?

ALGORITHM *Min1*($A[0..n - 1]$)

//Input: An array $A[0..n - 1]$ of real numbers

if $n = 1$ **return** $A[0]$

else $temp \leftarrow \text{Min1}(A[0..n - 2])$

if $temp \leq A[n - 1]$ **return** $temp$

else return $A[n - 1]$

ALGORITHM *Min2*($A[l..r]$)

if $l = r$ **return** $A[l]$

else $temp1 \leftarrow \text{Min2}(A[l..\lfloor (l + r)/2 \rfloor])$

$temp2 \leftarrow \text{Min2}(A[\lfloor (l + r)/2 \rfloor + 1..r])$

if $temp1 \leq temp2$ **return** $temp1$

else return $temp2$



1.4. Analysis of recursive algorithms

Your turn:

□ Abstract operation = ? Complexity = ?

Comparison

$$C_n = C_{n-1} + 1, n > 1$$
$$C_1 = 0$$

$$C_n = n - 1 = O(n)$$

ALGORITHM *Min1*($A[0..n - 1]$)

//Input: An array $A[0..n - 1]$ of real numbers

if $n = 1$ **return** $A[0]$

else $temp \leftarrow Min1(A[0..n - 2])$

if $temp \leq A[n - 1]$ **return** $temp$

else return $A[n - 1]$

ALGORITHM *Min2*($A[l..r]$)

if $l = r$ **return** $A[l]$

else $temp1 \leftarrow Min2(A[l..\lfloor (l + r)/2 \rfloor])$

$temp2 \leftarrow Min2(A[\lfloor (l + r)/2 \rfloor + 1..r])$

if $temp1 \leq temp2$ **return** $temp1$

else return $temp2$

$$C_n = 2C_{n/2} + 1, n > 1$$
$$C_1 = 0$$

$$C_n = n - 1 = O(n)$$



*** Average-case algorithm analysis

- For *average*-case algorithm analysis, we have to
 - characterize the inputs to the algorithm
 - calculate the *average* number of times the abstract operation is executed
 - calculate the *average* time complexity of the algorithm

But

- Average-case algorithm analysis requires detailed mathematical arguments.
- It's difficult to characterize the input data encountered in practice.



*** Average-case algorithm analysis

Steps for average-case analysis of an algorithm A :

1. Determine the sampling space which represents the *possible cases of input data* (of size n). Assume: the *sampling space* $S = \{I_1, I_2, \dots, I_k\}$.
2. Determine a probability distribution p in S which represents the *likelihood* that each case I_i ($i=1..k$) of the input data may occur.
3. Calculate the total number of times for abstract operation that the algorithm A executes to deal with a case I_i of input data in S . Let $v(I_i)$ denote the total number of times abstract operation is executed by the algorithm A when input data belong to the case I_i .
4. Calculate the average of the total number of times for abstract operation by using the following formula:

$$C_{avg}(n) = v(I_1).p(I_1) + v(I_2).p(I_2) + \dots + v(I_k).p(I_k).$$



*** Average-case algorithm analysis

Given an array A with n elements, find the location where the given value X occurs in array A .

```
Algorithm Find(A, n, i)
begin
  i := 1;
  while i <= n and X <> A[i] do
    i := i+1;
end
```

abstract
operation =
comparison

A	5	2	0	3	8	9	7	1	4	6
---	---	---	---	---	---	---	---	---	---	---

X	3
---	---

return → i = 4



*** Average-case algorithm analysis

In the case that *X is available in the array*, assume that the probability of the first match occurring in the i -th position of the array is the same for every i ($i=1..n$) and that probability is $p = 1/n$.

The number of comparisons to find X at the 1st position is 1.

The number of comparisons to find X at the 2nd position is 2.

...

The number of comparisons to find X at the n -th position is n

Therefore, the total number of comparisons in the average case is:

$$\begin{aligned} C_n &= 1.(1/n) + 2.(1/n) + \dots + n.(1/n) \\ &= (1 + 2 + \dots + n).(1/n) \\ &= (1+2+\dots+n)/n = n(n+1)/2.(1/n) = (n+1)/2 \end{aligned}$$



*** Average-case algorithm analysis

In the case that X is available in the array, assume that the probability of the first match occurring in the i -th position of the array is the same for every i ($i=1..n$) and that probability is $p = 1/n$.

The number of comparisons to find X at the 1st position is 1.

The number of comparisons to find X at the 2nd position is 2.

...

The number of comparisons to find X at the n -th position is n

Therefore, the total number of comparisons in the average case is:

$$\begin{aligned} C_n &= 1.(1/n) + 2.(1/n) + \dots + n.(1/n) \\ &= (1 + 2 + \dots + n).(1/n) \\ &= (1+2+\dots+n)/n = n(n+1)/2.(1/n) = (n+1)/2 = O(n) \end{aligned}$$



*** Average-case algorithm analysis

□ Your turn:

- Extension: Analyze the complexity of Find() algorithm in the average case when we include the situation where *X* is *not found* in *A*.
- Given the following program, what is its complexity in the average case if *comparison* is chosen as abstract operation? It is supposed that $n = 3$.

```
procedure MAXMIN(A, n, max, min)
/* Set max to the maximum and min to the minimum of A(1:n) */
begin
  integer i, n;
  max := A[1]; min := A[1];
  for i:= 2 to n do
    if A[i] > max then max := A[i]
    else if A[i] < min then min := A[i];
  end
```



*** Average-case algorithm analysis

□ Your turn:

- Given the following program, what is its complexity in the average case if *comparison* is chosen as abstract operation?

```
function UniqueElements (A, n)
begin
  for i:= 1 to n-1 do
    for j:= i + 1 to n do
      if A[i] = A[j] return false
  return true
end
```




*** Average-case algorithm analysis

□ Your turn:

- Given the following program, what is its complexity in the average case if *comparison* is chosen as abstract operation?

```
Algorithm closest-pair(X[1..n])  
// An array X[1..n] of n real numbers  
begin  
    Quicksort(X[1..n]);  
    for j := 2 to n do  
        D[j-1] := X[j] - X[j-1];  
    min := 1;  
    for j := 2 to n-1 do  
        if D[j] < D[min] then min := j;  
    writeln(X[min], X[min+1]);  
end;
```



1.5. Algorithm design strategies

- ❑ An **Algorithm Design Strategy** is a general approach to solving problems algorithmically that is applicable to a variety of problems from different areas of computing.
- ❑ Learning these strategies is very important for the following reasons:
 - They provide **guidance** for designing algorithms for new problems.
 - Algorithms are the **cornerstone** of computer science. Algorithm design strategies make it possible to classify and study algorithms.



1.5. Algorithm design strategies

- ❑ Brute force
 - Naïve string matching
- ❑ Divide-and-conquer
 - Quicksort
- ❑ Decrease-and-conquer
 - Insertion sort
- ❑ Transform-and-conquer
 - Heapsort
- ❑ Greedy approach
 - Graph coloring
- ❑ Dynamic programming
 - (0-1) Knapsack
- ❑ Backtracking
 - Knight's tour
- ❑ Branch and bound
 - Traveling salesman problem



1.6. Brute-force algorithm design

- ***Brute-force*** is a straightforward approach to solving a problem, usually directly based on the problem statement and definitions of the concepts involved.
 - ***"Just do it"***
 - the easiest to understand and the easiest to implement



1.6. Brute-force algorithm design

- String matching: find *all* the occurrences of a pattern P in a text T .
- A text T of n characters: $T[1..n]$
- A pattern P of m characters: $P[1..m]$
 - $m \leq n$
- A shift s shows that P occurs in T from $s+1$.
 - $T[s+1..s+m] = P[1..m]$ with $|T| = n$, $|P| = m$, and $0 \leq s \leq n - m$

Input: $T = \text{abcdefabaa}\text{def}$

$P1 = \text{def}$

Input: $T = \text{abcdefab}\text{aa}\text{def}$

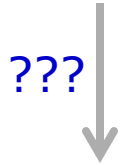
$P2 = \text{ab}$

Output: $P1$ occurs with shifts: 3, 10. Output: $P2$ occurs with shifts: 0, 6. 93



1.6. Brute-force algorithm design

Input: $T = \text{abcdefabaaef}$



$P1 = \text{def}$

Output: $P1$ occurs with shifts: 3, 10.

Input: $T = \text{abcdefabaaef}$



$P2 = \text{ab}$

Output: $P2$ occurs with shifts: 0, 6.

abcdefabaaef

Where is $P1$?

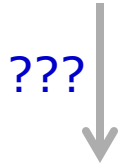
Where is $P2$?

Where is $P..$?



1.6. Brute-force algorithm design

Input: $T = \text{abcdefabaadeef}$



$P1 = \text{def}$

Output: $P1$ occurs with shifts: 3, 10.

Input: $T = \text{abcdefabaadeef}$



$P2 = \text{ab}$

Output: $P2$ occurs with shifts: 0, 6.

abcdefabaadeef

$s = 0$ $s = 1$ $s = 2$ $s = 3$ $s = 4$ FOUND!

$s = n - m$ $s = 1$ $s = n - m$

A sliding window of size m ($|P1|$)

Check if subtext matches $P1$ iteratively



1.6. Brute-force algorithm design

```
procedure NAIVE-STRING-MATCHING ( $T, P$ );  
begin  
   $n := |T|$ ;   $m := |P|$ ;  
  for  $s := 0$  to  $n - m$  do  
    if  $P[1..m] = T[s+1, \dots, s+m]$  then  
      print “Pattern occurs with shift”  $s$ ;  
end
```

Input: $T = \text{abcdefabaa}\text{def}$

$P1 = \text{def}$

Output: $P1$ occurs with shifts: 3, 10.



1.6. Brute-force algorithm design

```
procedure NAIVE-STRING-MATCHING (T, P);  
begin  
  n := |T|;  m := |P|;  
  for s := 0 to n - m do  
    begin  
      exit := false; k := 1;  
      while k ≤ m and not exit do  
        if P[k] ≠ T[s+k] then exit := true      //no matching  
        else k := k+1;  
      if not exit then  
        print “Pattern occurs with shift” s;  
    end  
  end
```



1.6. Brute-force algorithm design

□ Analysis of Naive-String-Matching

■ Step 1:

- Characterize the data: one input parameter n for T and one input parameter m for P
- Type of analysis: *all cases* (worst, average, best)

■ Step 2:

- Abstract operation: *comparison*

■ Step 3:

- *How many times is comparison done when the algorithm runs on T and P ?*

→ Your answer is ...



1.6. Brute-force algorithm design

```
procedure NAIVE-STRING-MATCHING (T, P);
```

```
begin
```

```
  n := |T|;  m := |P|;
```

```
  for s := 0 to n - m do
```

```
  begin
```

```
    exit := false; k := 1;
```

```
    while k ≤ m and not exit do
```

```
      if P[k] ≠ T[s+k] then exit := true      //no matching
```

```
      else k := k+1;
```

```
    if not exit then
```

```
      print "Pattern occurs with shift" s;
```

```
    end
```

```
end
```

Number of comparisons

= the total number of iterations

= **for** iterations * **while** iterations

= $(n-m+1) * m$

= $O((n-m+1)*m)$



1.6. Brute-force algorithm design

□ Analysis of Naive-String-Matching

■ Step 1:

- Characterize the data: one input parameter n for T and one input parameter m for P
- Type of analysis: *all cases* (worst, average, best)

■ Step 2:

- Abstract operation: *comparison*

■ Step 3:

- *How many times* is *comparison* done when the algorithm runs on T and P ?

→ Your answer is ... $(n-m+1)*m = O((n-m+1)*m)$



1.6. Brute-force algorithm design

□ Your turn:

- Design and analyze a *brute-force* algorithm for each following problem:
- Problem 1:
 - Given a weighted directed graph G of V vertices and E edges, find the least weighted path between vertex s and vertex t in G .
- Problem 2:
 - Given a list of T transactions each of which is made by a customer, a transaction includes some items in I , which is a finite list of all the items distinct in T . You are asked to find all the 3-item combinations that are bought by at least 50% customers.

Summary

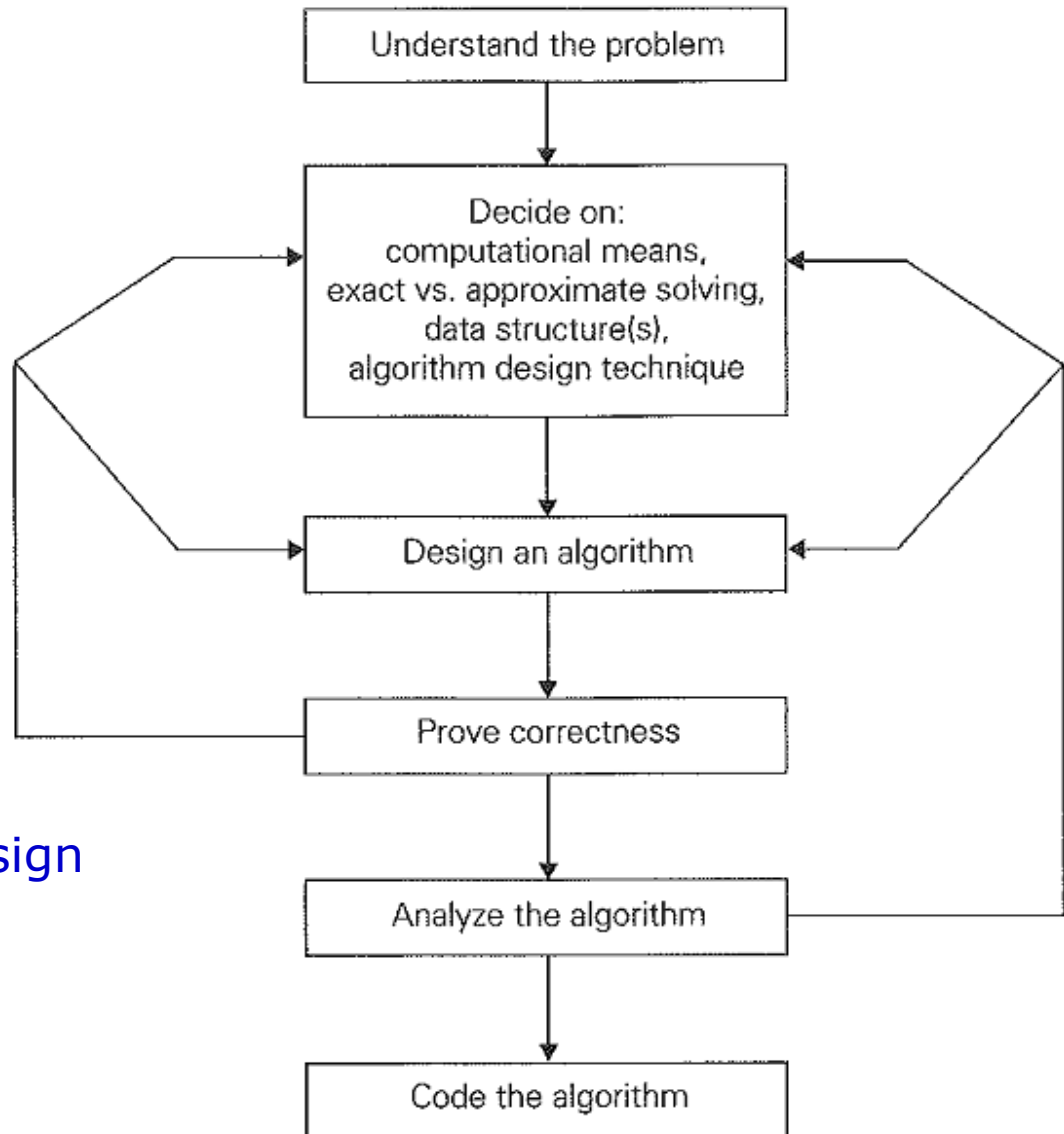


Figure 1.2. Algorithm design
and analysis process

[2], pp. 10.



Algorithms Analysis and Design (C03031)

□ Chapter 1. Fundamentals



- 1.1. Recursion and recurrence relations
- 1.2. Analysis of algorithms
- 1.3. Analysis of iterative algorithms
- 1.4. Analysis of recursive algorithms
- 1.5. Algorithm design strategies
- 1.6. Brute-force algorithm design